Choice Healthcare
Patient Pipeline Tracker

Architecture, AWS & Database Report

Generated: June 08, 2026

Version 1 (tracker only - no document storage)

1. Executive Summary

Choice Healthcare built an internal web application to replace scattered spreadsheets and email chains for tracking custom power-wheelchair prior-authorization workflows. Each patient gets a shared, prioritized checklist of tasks. Reps drive the workflow forward; ATPs (Assistive Technology Professionals) approve gated clinical tasks; Managers see their team's patients; the Boss has full visibility. The app does not store documents in v1 - tasks may link to external URLs (e.g. Google Drive).

The system is split across two AWS services: AWS Amplify hosts the Next.js web application, and an AWS EC2 instance hosts a self-hosted open-source Supabase stack (Postgres 17, GoTrue Auth, PostgREST, Kong API gateway). The database is NOT on your local development computer - it runs in the cloud on EC2.

2. What Was Built

Application features (v1)

- Microsoft/Azure OAuth login (primary for production) plus email/password for dev/pilot users.
- Role-based visibility: REP, ATP, MANAGER, BOSS - stored as a text array on each user profile.
- Patient list and detail pages with a full per-patient task checklist snapshotted from templates.
- New-patient form that atomically creates a patient row and instantiates tasks from payer-type templates.
- Dashboard priority queue that surfaces the most urgent open tasks across all visible patients.
- Inline task editing: status, due dates, priority, links, blocked reasons, and 'Sent for signature' status.
- ATP approval gate enforced in the database - only the assigned ATP (or Boss, or solo-case rep/ATP) can approve gated tasks.
- Admin panel: activate users, assign roles, manage payer types, edit task templates, create/delete accounts.
- Task link history table tracking each document URL submission per task.

Technology stack

- Frontend: Next.js 16.2 (App Router), React 19.2, Tailwind CSS 4.
- Backend data layer: Supabase open-source - Postgres 17 + Row-Level Security (RLS).
- Client libraries: @supabase/ssr for cookie-based auth in server components and browser.
- Auth middleware: src/proxy.ts refreshes session cookies and redirects unauthenticated users to /login.

3. AWS Architecture

Production uses a two-tier AWS layout. The website and the database are separate resources with different update paths.

Component	AWS Service	Role
Web application	AWS Amplify	Hosts Next.js UI; rebuilds on git push to main
Database + Auth API	EC2 (Elastic IP)	Self-hosted Supabase in Docker at /opt/choice-supabase

Production identifiers

Item	Value
Live app URL	https://main.d2na0dxbmaa2o4.amplifyapp.com
Amplify app ID	d2na0dxbmaa2o4
AWS region	us-west-2 (US West, Oregon)
EC2 instance ID	i-0c55b5678f0ec6cf7
EC2 public IP (Elastic IP)	44.253.198.43
Instance type	t4g.small, Ubuntu 22.04, 30 GB encrypted EBS
Security group	sg-07721f9fedc45d2fa (choice-tracker-sg)
Supabase API (Kong)	http://44.253.198.43:8000
Supabase install path	/opt/choice-supabase
Postgres port	5432 - internal to Docker only; NOT open to the public internet

Request flow

Browser -> AWS Amplify (HTTPS Next.js app)
-> For production: Next.js rewrites /supabase/* to EC2 Kong gateway (HTTP :8000)
-> Kong routes to GoTrue (auth) or PostgREST (database API)
-> Postgres 17 stores all patient/task/user data with RLS policies

This HTTPS proxy pattern exists because browsers block HTTPS pages from calling plain HTTP APIs (mixed-content policy). Amplify sets SUPABASE_INTERNAL_URL to the EC2 address and NEXT_PUBLIC_SUPABASE_URL to https://<amplify-app>/supabase so all API traffic is same-origin HTTPS.

What updates when you change things

- UI changes, buttons, labels -> git push -> Amplify auto-rebuilds (minutes).
- Schema changes (new columns, statuses, tables, RLS) -> run SQL migrations on EC2 (manual, via SSH or browser terminal).
- App env vars (Supabase URL, keys) -> AWS Amplify Console environment variables.

4. Where Is the Database Hosted?

The authoritative database is hosted on the AWS EC2 instance at 44.253.198.43. It runs inside Docker containers as part of the self-hosted Supabase stack cloned to /opt/choice-supabase on that server.

Is Supabase on this computer?

No - not for production or current local development on this machine. Findings from this workstation:
- Supabase CLI is NOT installed (supabase command not found).
- Docker Desktop is NOT running (no local containers).
- .env.local points NEXT_PUBLIC_SUPABASE_URL to http://44.253.198.43:8000 (remote EC2).

The repo includes supabase/config.toml and supabase/migrations/ for local development via 'supabase start' (ports 54321/54322/54323), but that local stack is optional and is not currently active on this machine. When used, it would run Postgres in Docker on localhost - separate from the production EC2 database.

Retired hosting

An earlier managed Supabase Cloud project (ftxxexwzrhyrqjguagbi.supabase.co) was used during initial development with synthetic seed data. The project has been superseded by self-hosted Supabase on EC2 to avoid managed HIPAA-tier costs. Do not use the cloud project for new deployments.

5. API Keys and Secrets

This section describes which keys exist and where they are used. Actual secret values are NOT included in this report - they live in environment files and the EC2 server, never in git.

Application environment variables

Variable	Exposure	Purpose
NEXT_PUBLIC_SUPABASE_URL	Public (browser)	Supabase API base URL. Local dev: EC2 :8000. Prod: Amplify
NEXT_PUBLIC_SUPABASE_ANON_KEY	Public (browser)	JWT anon key - authenticates as 'authenticated' role. Respects
SUPABASE_SERVICE_ROLE_KEY	Server-only	Bypasses RLS. Used for admin.auth.createUser/deleteUser onl
SUPABASE_INTERNAL_URL	Server-only (Amplify)	EC2 Kong URL for Next.js rewrite proxy in production.
NEXT_PUBLIC_APP_URL	Public	Base URL for OAuth callbacks and sign-out redirects.
NEXT_PUBLIC_AUTH_*_ENABLED	Public	Feature flags for Azure, Google, email login providers.
SUPABASE_AUTH_EXTERNAL_AZURE_*	Server/Supabase	Microsoft Entra OAuth client ID and secret.

EC2 Supabase stack keys (/opt/choice-supabase/.env)

- ANON_KEY - same JWT anon key surfaced to the Next.js app as NEXT_PUBLIC_SUPABASE_ANON_KEY.
- SERVICE_ROLE_KEY - same as SUPABASE_SERVICE_ROLE_KEY in Amplify/server env.
- JWT_SECRET - signs auth tokens inside GoTrue; never sent to the browser.
- POSTGRES_PASSWORD - database superuser password; used only inside Docker.
- SITE_URL / API_EXTERNAL_URL / SUPABASE_PUBLIC_URL - OAuth redirect configuration.

Current workstation (.env.local) status

- NEXT_PUBLIC_SUPABASE_URL = http://44.253.198.43:8000 (remote EC2, not localhost).
- NEXT_PUBLIC_SUPABASE_ANON_KEY = configured (212-character JWT).
- SUPABASE_SERVICE_ROLE_KEY = not set in .env.local (admin user create/delete requires it).
- NEXT_PUBLIC_AUTH_EMAIL_ENABLED = true; Azure and Google = false.

AWS deployment keys (not in application code)

- IAM access keys - used by developers for AWS CLI (EC2 launch, Amplify env updates). Rotate after bootstrap.
- SSH key pair choice-tracker-key.pem - stored at %USERPROFILE%\.ssh\ for EC2 admin access.
- These are infrastructure credentials, separate from Supabase JWT keys.

6. Database Schema

Postgres 17 database 'postgres', schema 'public'. All tables have Row-Level Security enabled. Eleven migration files

(0001 through 0011) define the current schema.

Core tables

Table	Description
app_users	User profiles linked 1:1 to auth.users. Roles[], manager_id, supervising_atp_id, active flag.
payers	Insurance payers (name + type). Type FK references payer_types.code.
payer_types	Admin-managed workflow categories: MEDICARE, MEDICAID, COMMERCIAL (+ custom).
patients	Patient + active case merged. Names, payer, assigned rep/ATP, pursuit status.
task_templates	Master checklist per payer type. Label, role, ATP-review flag, order.
tasks	Per-patient instantiated checklist. Snapshotted from template; holds status, dates, link.
task_link_events	History of link submissions per task (who posted, when, URL or 'via other means').

Entity relationships

app_users.manager_id -> app_users (org hierarchy)
app_users.supervising_atp_id -> app_users (default ATP for reps)
patients.payer_id -> payers.id
payers.type -> payer_types.code
patients.assigned_rep_id / assigned_atp_id -> app_users.id
tasks.patient_id -> patients.id (cascade delete)
tasks.template_id -> task_templates.id (set null on delete)
task_link_events.task_id -> tasks.id (cascade delete)

Allowed status and role values

- User roles: ATP, REP, MANAGER, BOSS (multi-valued array).
- Patient status: ACTIVE, SUBMITTED, APPROVED, DENIED, DELIVERED, CLOSED.
- Task status: NOT_STARTED, IN_PROGRESS, AWAITING_SIGNATURE, DONE_PENDING_REVIEW, APPROVED, BLOCKED.
- Task responsible_role: DOCTOR, PT, ATP, REP, FRONT_DESK.
- Payer types: dynamic via payer_types table (seeded: COMMERCIAL, MEDICAID, MEDICARE).

Key business rules in the database

- Tasks are snapshotted at patient creation - editing templates does not rewrite in-flight patients.
- RLS policies control row visibility by role and patient assignment.
- enforce_task_approval_gate trigger blocks unauthorized APPROVED transitions on ATP-gated tasks.
- create_patient_with_tasks() RPC atomically creates patient + all template tasks.
- update_app_user() RPC is the only way to change user profiles (direct table updates disabled).
- on_auth_user_created trigger auto-creates app_users row (REP, inactive) on first OAuth sign-in.

7. Supabase Services on EC2

The Docker stack is trimmed for v1 - unnecessary Supabase services are disabled:

- Enabled: db (Postgres 17), auth (GoTrue), rest (PostgREST), kong (API gateway), meta.

- Disabled: storage, realtime, analytics, edge functions, imgproxy, pooler - v1 has no file uploads or push.

Kong listens on port 8000 (public via security group). PostgREST exposes the public schema over REST. GoTrue handles OAuth and email/password sessions. Studio is disabled in production override.

8. Authentication Flow

1. User visits any page -> proxy.ts checks Supabase session cookie.
2. No session -> redirect to /login.
3. Login via Microsoft OAuth or email/password -> Supabase GoTrue on EC2.
4. OAuth callback at /auth/callback exchanges code for session cookie.
5. First sign-in triggers handle_new_auth_user -> app_users row (REP, inactive).
6. Admin activates user and assigns roles in /admin.
7. All subsequent data access uses the anon key + user JWT; RLS enforces permissions.

9. Migration History

- 0001_init.sql - Core tables, indexes, handle_new_auth_user trigger.
- 0002_rls.sql - Security-definer helpers + RLS policies on all tables.
- 0003_approve_gate.sql - ATP approval gate trigger + completion stamps.
- 0004_supervising_atp.sql - supervising_atp_id on app_users.
- 0005_harden_user_and_patient_workflows.sql - update_app_user RPC, create_patient_with_tasks RPC.
- 0006_fix_create_patient_payer_type.sql - Payer type lookup fix in patient creation.
- 0007_task_link_history.sql - task_link_events table + RLS.
- 0008_requires_atp_review_default_true.sql - Default ATP review flag on templates.
- 0009_payer_types_admin.sql - payer_types table; FK replaces hard-coded CHECK constraints.
- 0010_ensure_builtin_payer_types.sql - Seed built-in payer type rows.
- 0011_task_awaiting_signature_status.sql - AWAITING_SIGNATURE task status.

10. Security & HIPAA Notes

- v1 intentionally avoids document storage to sidestep HIPAA-tier Supabase Cloud costs.
- Current EC2 stack uses synthetic seed data for validation - real PHI requires AWS BAA, backups, audit logging.
- Postgres port 5432 is not exposed to the public internet; migrations run inside Docker on EC2.
- Never commit .env.local, .env.aws.local, or PEM keys to git.
- Disable email signup and dev password auth before production cutover with real patients.
- Do not log patient names (PHI) - log patient IDs and external codes only.

11. Quick Reference - Common Operations

- Apply DB migrations: infra/aws/scripts/apply-migrations-from-windows.ps1 or EC2 Instance Connect browser terminal.
- Read EC2 keys: SSH to EC2, sudo grep ANON_KEY /opt/choice-supabase/.env

- Update Amplify env: infra/aws/scripts/update-amplify-env.ps1
- Local dev: npm run dev with .env.local pointing at EC2 (current setup) or supabase start for fully local.
- Demo login (seed data): tara@choice.example / password123

Sources: ARCHITECTURE.md, infra/aws/DEPLOYMENT.md, CLAUDE.md, supabase/migrations/, .env.example, and live inspection of this development workstation (June 2026).